Full Stack Development With AI

Raghavendran V

Saturday, August 30, 2025
11:00 AM SGT

NUS National University of Singapore | Advanced Computing for Executives
School of Computing

Agenda

- Introduction to Frontend Frameworks
- Compiler + React
- What is Node.js?
- Component-Based Design
- What is a React SPA?

Introduction to Frontend Frameworks

1. What is a Frontend Framework?

A **frontend framework** is a pre-written set of code (libraries + tools) that helps developers build the **user interface (UI)** of a web application more efficiently.

Instead of writing everything from scratch in HTML, CSS, and JavaScript, you can use a framework that provides:

- Ready-made **components** (buttons, forms, grids, modals, etc.)
- Efficient ways to **manage state** (data in your app)
- Tools for **routing** (navigating between pages)
- Support for **responsive design**
- Faster development and easier maintenance

2. Popular Frontend Frameworks

a) React.js (Library)

- Created by **Facebook (Meta)**
- Based on **components** and **Virtual DOM**
- Focuses only on the **view layer** (needs external libraries for routing, state management, etc.)
- Example:

```
function App() {
  return <h1>Hello, React!</h1>;
}
```

b) Angular

- Developed by **Google**
- Full-fledged framework (has built-in routing, HTTP, forms, etc.)
- Uses **TypeScript** by default
- Example:

```
<h1>{{ title }}</h1>
export class AppComponent {
  title = 'Hello Angular!';
}
```

c) Vue.js

- Lightweight and flexible
- Easy learning curve compared to Angular
- Supports both small and large-scale applications
- Example:

```
<div id="app">
  <h1>{{ message }}</h1>
</div>
<script>
new Vue({
  el: '#app',
  data: {
    message: 'Hello Vue!'
  }
});
</script>
```

Comparison Table

Feature	React	Angular	Vue
Type	Library	Framework	Framework
Language	JS/TS	TypeScript	JS/TS
Learning Curve	Medium	Steep	Easy
Performance	High	Moderate	High
Ecosystem	Large	Complete	Growing

Compiler + React

1. Does React use a Compiler?

React itself is a **JavaScript library**, so the browser directly runs it without needing a "traditional compiler" (like C/C++ → machine code).

But modern React development **does involve compilation and build tools** that act like compilers for JavaScript, JSX, and CSS.

2. Where Compiler Fits in React?

When we write React code like this:

```
function App() {
  return <h1>Hello React!</h1>;
}
```

The browser **does not understand JSX** (`<h1>...</h1>`).
So tools like **Babel (compiler)** convert it into **plain JavaScript**:

```
function App() {
  return React.createElement("h1", null, "Hello React!");
}
```

✅ That’s compilation happening in React development!

3. Tools that Act as "Compilers" in React

- **Babel** → Translates modern JavaScript (ES6+, JSX) → older JavaScript browsers can understand.

- **TypeScript Compiler (tsc)** → Converts TypeScript → JavaScript.

- **Webpack / Vite / Parcel** → Bundlers that compile, optimize, and package code into files for the browser.

- **SWC / esbuild** → Modern compilers (faster alternatives to Babel).

4. Example Flow: React App Build

1. **Developer writes**:

   ```
   const App = () => <h2>Welcome!</h2>;
   ```

2. **Babel compiles JSX → JS**:

   ```
   const App = () => React.createElement("h2", null, "Welcome!");
   ```

3. **Bundler (Webpack/Vite) compiles** all JS, CSS, images into a single optimized `bundle.js`.

4. **Browser executes**:
   Runs the plain JavaScript bundle → renders `<h2>Welcome!</h2>` on screen.

5. Why is Compilation Important in React?

- JSX → Browser doesn’t understand JSX, so Babel compiles it.
- Modern JavaScript (ES6+) → Compiled for older browsers.
- Performance → Bundlers optimize and minify code for faster apps.
- Type Safety → TypeScript compiler ensures fewer bugs.

6. Quick Analogy

- **Traditional Compiler (C/C++)** → Converts code → machine code.
- **React’s "Compiler" Tools** → Convert JSX/TS/ES6 → browser-compatible JavaScript.

Component-Based Design

1. What is Component-Based Design?

It's a **software design approach** where an application is built using **reusable, independent pieces** called **components**.

Each **component**:

- Encapsulates UI + logic
- Can be reused across the app
- Communicates with other components via **props** or **events**

2. Why Components?

✅ **Reusability** → Build once, use everywhere.
✅ **Maintainability** → Easier to fix/update one component than a big file.
✅ **Separation of concerns** → UI, logic, and styling live inside the component.
✅ **Scalability** → Large apps can be split into smaller, manageable parts.

3. React Components Types

a) Functional Component (most common today)

```
function Greeting(props) {
  return <h1>Hello, {props.name}!</h1>;
}
```

Usage:

```
<Greeting name="Raghav" />
<Greeting name="Alex" />
```

b) Class Component (older style)

```
class Greeting extends React.Component {
  render() {
    return <h1>Hello, {this.props.name}!</h1>;
  }
}
```

4. Real-World Analogy

Think of a **Car** 🚗 as a system:

- Engine → Component
- Wheels → Component
- Headlights → Component
- Dashboard → Component

Each part is independent but works together to form the car. Similarly, components work together to form an app.

5. Example: Component-Based UI

```
// Button Component
function Button({ label }) {
  return <button>{label}</button>;
}

// Header Component
function Header() {
  return <h1>Welcome to My App</h1>;
}

// App Component (composed of smaller components)
function App() {
  return (
    <div>
      <Header />
      <Button label="Login" />
      <Button label="Signup" />
    </div>
  );
}
```

🖥️ Output:

```
Welcome to My App
[Login] [Signup]
```

6. How Components Work Together

- **Parent Component** → Holds other components.
- **Child Components** → Receive data (props) and render UI.
- **State** → Managed within a component or shared via context.

✅ **Summary**:
Component-Based Design breaks an application into **small, reusable, self-contained building blocks**.
React apps follow this approach heavily — making them **modular, maintainable, and scalable**.

What is Node.js?

Node.js is an **open-source, cross-platform JavaScript runtime environment** that allows developers to run JavaScript code **outside of the browser**.

👉 In simple words:

- Normally, JavaScript runs only inside web browsers (like Chrome, Firefox).
- **Node.js lets you run JavaScript on your computer/server** — just like Python, Java, or C++.

It was built on **Google Chrome's V8 JavaScript Engine** (the same engine powering Chrome).

⚡ Why Node.js is Important?

Before Node.js:

- JavaScript was **client-side only** (ran in browsers).
- Backend development used languages like Python, Java, PHP, etc.

With Node.js:

- You can use **JavaScript everywhere** (Frontend + Backend).
- One language across the full stack (**Full Stack Development**).

🔄 Node.js vs Browser JavaScript

Feature	Browser JS	Node.js
Runs in	Browser	Server/PC
Access to DOM	✅ Yes	❌ No
Access to Filesystem	❌ No	✅ Yes
Use Case	User Interfaces	Backend, APIs, Servers

What is a React SPA?

- **SPA = Single Page Application**
- In a **traditional website**, every time you click a link, the browser **requests a new page** from the server.
- In an **SPA**, the server usually sends a **single HTML page** (often `index.html`), and **React takes over navigation** on the client side.

This means:

- No full page reloads.
- Navigation is handled by **JavaScript + React Router**.
- Data/content is fetched dynamically (often from APIs).

How React SPA Works

1. **Initial load**: Server sends `index.html` with bundled JS/CSS.

2. **React loads** in the browser → mounts the root component (`<App />`).

3. **Routing**: Instead of requesting a new page, React swaps components inside the same page (`<Home />`, `<About />`, `<Profile />`).

4. **API Calls**: React fetches different data for different views.

Example React SPA with Routing

```
// App.js
import React from "react";
import { BrowserRouter as Router, Routes, Route, Link } from "react-router-dom";

function Home() {
  return <h2>🏠 Home Page</h2>;
}

function About() {
  return <h2>ℹ️ About Page</h2>;
}

export default function App() {
  return (
    <Router>
      <nav>
        <Link to="/">Home</Link> |
        <Link to="/about">About</Link> |
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </Router>
  );
}
```

Benefits of React SPA

- ✅ **Fast navigation** (no reload).
- ✅ **Smooth user experience** like native apps.
- ✅ **Component reusability**.
- ✅ **API-driven** (fetch data without reloading).

LIVE SESSION EXPERIENCE SURVEY

Before we proceed to Q&A
Take 2 minutes to share your feedback with Us!